Established – 1961 Subject: **--- OOPS ---**

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE ULHASNA-GAR – 421 003

CERTIFICATE

This is to certify that Mr./Ms. Ritika Santosh Kumar Kori of F.Y. Information Technology (FYIT) Roll No. 2541017 has satisfactorily completed the Web Designing Mini Project entitled Hotel Check-In/ Check-Out System during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

Kumodh Kukreja **Laxmi Jheswani**

INDEX

Topic: Hotel Check-In / Check-Out System.

1. Introduction.

The **Hotel Check-In and Check-Out System** is a consolebased application developed using the **C++** programming language by applying the principles of **Object Oriented Programming (OOP)**. Hotels require a systematic and reliable solution to manage guest details, room allocation, check-in, check-out, and billing processes efficiently.

This project is designed using core **OOP** concepts such as **Encapsulation**, **Hierarchical Inheritance**, **Polymorphism**, and **Abstract Classes**. These concepts help in organizing the program into reusable and manageable components. The system also makes use of pointers for dynamic memory handling and **runtime polymorphism** using virtual functions.

2. Topic Description.

The **Hotel Check-In and Check-Out System** is a console-based application developed using the **C++ programming language**. It is designed to manage hotel operations such as guest check-in, room allocation, check-out, and bill calculation.

The project follows **Object Oriented Programming** concepts including Encapsulation, **Hierarchical Inheritance**, **Polymorphism,** and **Abstract Classes**. These concepts help in organizing the program into reusable and secure components. Runtime **polymorphism** and pointers are used to improve flexibility and efficient memory management. This system demonstrates the practical use of **OOPs** concept in a real-world scenario**.**

3. OOPS Concept Used.

- **Encapsulation – Data Hiding.**

The project uses data hiding type of encapsulation.

All data members are declared as private inside the derived class.

Access to these variables is provided only through public member functions.

This restricts direct access to data and ensures controlled manipulation.

- **Inheritance – Single Inheritance.**

The project uses Single Inheritance.

One derived class (RoomBooking) inherits from one base class (Hotel).

There is only one parent class and one child class, so it is not multiple or multilevel inheritance.

- **Abstract Class – Pure Abstract Class.**

The base class is a Pure Abstract Class because it contains only pure virtual functions.

It acts as a blueprint and cannot be instantiated.

The derived class provides the implementation of those virtual functions.

- **Polymorphism – Runtime Polymorphism.**

The project implements Runtime Polymorphism.

This is achieved using:

Virtual functions

Base class pointer referring to derived class object

The function call is resolved at runtime, not at compile time.

- **Static Data Members – Class Level Static Members.**

The project uses static data members at class level.

These static variables are shared among all objects of the class and are used for automatic room number generation.

4. Implementation Code.

```
#include <iostream>
using namespace std;

class Hotel
{
public:
    virtual void bookRoom() = 0;
    virtual void generateBill() = 0;
    virtual ~Hotel() {}
};

class RoomBooking : public Hotel
{
private:
    string guestName;
    int roomNumber;
    int roomTypeChoice;
    int numberOfRooms;
    int numberOfPeople;
    int days;
    float roomRate;
    float extraServiceCharges;
    float totalBill;

    // Room Counters (Room Numbers)
    static int singleRoomCounter;
    static int standardRoomCounter;
```

```
    static int deluxeRoomCounter;
    static int suiteRoomCounter;
    static int presidentialRoomCounter;

    // Room Availability
    static int singleAvailable;
    static int standardAvailable;
    static int deluxeAvailable;
    static int suiteAvailable;
    static int presidentialAvailable;

public:
    void bookRoom()
    {
cout<<"=======================================\n";
cout << "       GRAND PALACE HOTEL\n";
cout<<"=======================================\n";

        cout << "Enter Guest Name: ";
        cin >> guestName;

        while (true)
        {
            cout << "\nSelect Room Type:\n";
            cout << "1. Single        (800 per day)\n";
            cout << "2. Standard      (1000 per day)\n";
            cout << "3. Deluxe        (2000 per day)\n";
            cout << "4. Suite         (3500 per day)\n";
            cout << "5. Presidential  (5000 per day)\n";
            cout << "Enter Choice (1-5): ";
```

```
        cin >> roomTypeChoice;

        if (roomTypeChoice >= 1 && roomTypeChoice <= 5)
            break;
        else
            cout << "Invalid choice! Please enter between 1 and
5.\n";
    }

    bool available = true;

    switch(roomTypeChoice)
    {
        case 1:
            if(singleAvailable > 0)
            {
                roomRate = 800;
                roomNumber = singleRoomCounter++;
                singleAvailable--;
            }
            else
            {
                cout << "Single Rooms are not available!\n";
                available = false;
            }
            break;

        case 2:
            if(standardAvailable > 0)
            {
```

```
            roomRate = 1000;
            roomNumber = standardRoomCounter++;
            standardAvailable--;
        }
        else
        {
            cout << "Standard Rooms are not available!\n";
            available = false;
        }
        break;

    case 3:
        if(deluxeAvailable > 0)
        {
            roomRate = 2000;
            roomNumber = deluxeRoomCounter++;
            deluxeAvailable--;
        }
        else
        {
            cout << "Deluxe Rooms are not available!\n";
            available = false;
        }
        break;

    case 4:
        if(suiteAvailable > 0)
        {
            roomRate = 3500;
            roomNumber = suiteRoomCounter++;
```

```
            suiteAvailable--;
        }
        else
        {
            cout << "Suite Rooms are not available!\n";
            available = false;
        }
        break;

    case 5:
        if(presidentialAvailable > 0)
        {
            roomRate = 5000;
            roomNumber = presidentialRoomCounter++;
            presidentialAvailable--;
        }
        else
        {
            cout << "Presidential Room is not available!\n";
            available = false;
        }
        break;
}

if(!available)
    return;

cout << "Room Number: " << roomNumber << endl;

cout << "Enter Number of Rooms: ";
```

```
        cin >> numberOfRooms;

        cout << "Enter Number of People: ";
        cin >> numberOfPeople;

        cout << "Enter Number of Days: ";
        cin >> days;

        cout << "Enter Extra Service Charges: ";
        cin >> extraServiceCharges;

        cout << "\nRoom Booked Successfully!\n\n";
    }

    void generateBill()
    {
        totalBill = (roomRate * numberOfRooms * days) +
extraServiceCharges;

cout<<"\n=======================================
=======\n";
cout << "              GRAND PALACE HOTEL\n";
cout << "           Luxury | Comfort | Trust\n";
cout<<"=======================================
======\n";

        cout << "\n========== HOTEL BILL ==========\n";
        cout << "Guest Name: " << guestName << endl;
        cout << "Room Number: " << roomNumber << endl;
```

```
        cout << "Room Type: ";
        if(roomTypeChoice == 1) cout << "Single\n";
        else if(roomTypeChoice == 2) cout << "Standard\n";
        else if(roomTypeChoice == 3) cout << "Deluxe\n";
        else if(roomTypeChoice == 4) cout << "Suite\n";
        else if(roomTypeChoice == 5) cout << "Presidential\n";

        cout << "Number of Rooms: " << numberOfRooms << endl;
        cout << "Number of People: " << numberOfPeople << endl;
        cout << "Days Stayed: " << days << endl;

        cout << "Room Charges: " << roomRate *
numberOfRooms * days << endl;
        cout << "Extra Charges: " << extraServiceCharges << endl;
        cout << "Total Bill: " << totalBill << endl;
        cout                                                    <<
"=============================================================
=\n";
    }
};

// Initial Room Numbers
int RoomBooking::singleRoomCounter = 302;
int RoomBooking::standardRoomCounter = 421;
int RoomBooking::deluxeRoomCounter = 153;
int RoomBooking::suiteRoomCounter = 309;
int RoomBooking::presidentialRoomCounter = 251;

// Initial Availability
int RoomBooking::singleAvailable = 5;
```

```
int RoomBooking::standardAvailable = 5;
int RoomBooking::deluxeAvailable = 3;
int RoomBooking::suiteAvailable = 0;
int RoomBooking::presidentialAvailable = 1;

int main()
{
   Hotel* booking;
   RoomBooking rb;

   booking = &rb;

   booking->bookRoom();
   booking->generateBill();

   return 0;
}
```

5. Output.

1.)

```
==========================================
            GRAND PALACE HOTEL
==========================================
Enter Guest Name: Ryna

Select Room Type:
1. Single       (800 per day)
2. Standard     (1000 per day)
3. Deluxe       (2000 per day)
4. Suite        (3500 per day)
```

5. Presidential (5000 per day)
Enter Choice (1-5): 3
Room Number: 153
Enter Number of Rooms: 6
Enter Number of People: 6
Enter Number of Days: 3
Enter Extra Service Charges: 4500

Room Booked Successfully!

==========

GRAND PALACE HOTEL

Luxury | Comfort | Trust

==========

========== HOTEL BILL ==========

Guest Name: Ryna
Room Number: 153
Room Type: Deluxe
Number of Rooms: 6
Number of People: 6
Days Stayed: 3
Room Charges: 36000
Extra Charges: 4500
Total Bill: 40500

==========

=== Code Execution Successful ===

2.)

==========

GRAND PALACE HOTEL

==========

Enter Guest Name: Kirya

Select Room Type:

1. Single (800 per day)

2. Standard (1000 per day)

3. Deluxe (2000 per day)

4. Suite (3500 per day)

5. Presidential (5000 per day)

Enter Choice (1-5): 4

Suite Rooms are not available!

==========

3.)

==========

GRAND PALACE HOTEL

==========

Enter Guest Name: Shree

Select Room Type:

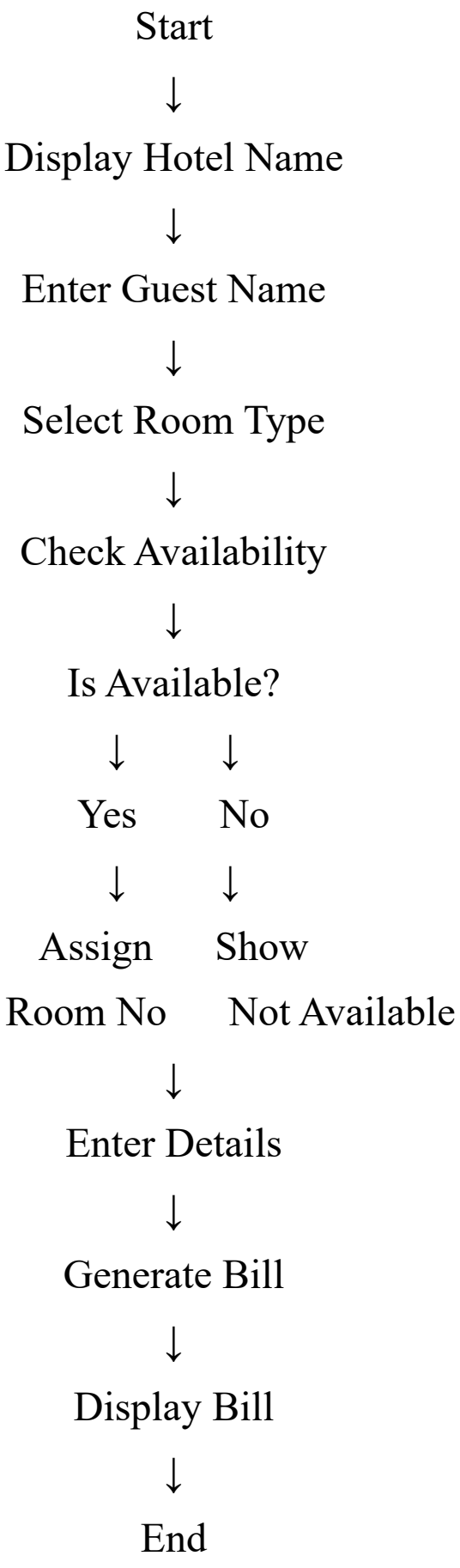
1. Single (800 per day)
2. Standard (1000 per day)
3. Deluxe (2000 per day)
4. Suite (3500 per day)
5. Presidential (5000 per day)

Enter Choice (1-5): 6

Invalid choice! Please enter between 1 and 5.

==========

6. Diagram


Start
Display Hotel Name
Enter Guest Name
Select Room Type
Check Availability
Is Available?
Yes
No
Assign Room No
Show Not Available
Enter Details
Generate Bill
Display Bill
End

7. Conclusion.

The Hotel Management System project was developed using Object-Oriented Programming concepts in C++. This system helps manage room bookings, calculate bills, and assign room numbers automatically based on room types.

In this project, important OOP concepts such as Abstraction, Inheritance, Polymorphism, and Encapsulation were successfully implemented. The abstract base class Hotel ensures abstraction by defining pure virtual functions. The RoomBooking class inherits from it and implements the required functionalities. Polymorphism is achieved using a base class pointer, and encapsulation is maintained by keeping data members private.

The system allows users to:

- Select different room types
- Book rooms
- Add extra service charges
- Generate detailed bills

This project improves understanding of real-world application of OOP concepts and demonstrates how programming can be used to automate hotel management tasks efficiently.